



Design, Implement and Test a Procedural Programming Solution

ESOFT UNIVERSITY
BACHELOR OF INFORMATION TECHNOLOGY (HONOURS)
CONTINUOUS ASSESSMENT
CCU1304 – FUNDAMENTALS OF PROGRAMMING

Name: M.W.O.C.Wickramasinghe.

Registration Number: E310875

Module Lecturer: Ms.Thilini.

Introduction

This report presents the QuickBite Food Ordering System, a Python console application designed to automate order calculations, membership discounts, and distance-based delivery fees. The primary objective is to deliver a reliable, crash-free application through robust input validation and error handling.

Table of Contents

1. Problem Analysis.....	4
1.1 Introduction.....	4
1.2 Inputs.....	4
1.3 Processing	4
1.4 Outputs.....	5
1.5 Assumptions.....	5
1.6 Constrains	5
2. Algorithm Design.....	6
2.1 Algorithm.....	6
2.2 Psuedocode	8
2.3 Flowchart	8
4. Program.....	10
4.1 Constants and Functions.	10
4.2 User Inputs and Data Validation.....	11
4.3 Calculations.....	12
4.4 Generating the Itemized Receipt.....	13
5. Input Validations.....	14
5.1 How Input Validation Works in this Program	14
5.1.1 Order Type Validation	14
5.1.2 Number of Items Validation.....	15
5.1.3 Price per Item Validation	16
5.1.4 Delivery Distance Validation.....	17
5.1.5 Discount Eligibility Validation	18
6. Testing.....	20
6.1 Test Plan.....	20
6.2 Test Cases	21
7. Test Evidence.....	22
TC01	22
TC02	22
TC03	23

TC04	23
TC05	24
TC06	24
TC07	24
TC08	25
8. Technical Documentation.	25
8.1. Main Functions.	25
1. Validate input ()	25
2. Calculate bill ().....	26
3. Display bill ()	26
8.2 Variables Used	26
8.3 Programming Decisions.	27
9. Reflection.	27
9.1 Future Enhancements.	27

1. Problem Analysis.

1.1 Introduction

For **QuickBite**, there is a need for an automated console billing application to avoid any manual errors and inefficiencies during the process. The billing process will be carried out using the following procedural python program.

1.2 Inputs

Input	Data Type	Description
Customer Name	String	Name of the customer
Order Type	String	Regular, Premium, VIP
Number of Food Items	Integer	Total quantity ordered
Price per Item	Integer	Price of one food item
Delivery distance	Float	Distance in kilometers
Discount eligibility	String	Eligible for promo discount(Y/N)

1.3 Processing

The program performs the following operations:

I. Read all user inputs.

II. Validate the entered data.

III. Calculate Food Subtotal.

$\text{Food Subtotal} = \text{Number of items} * \text{Price per Items}$

IV. Calculate delivery charge.

$\text{Delivery Charge} = \text{Distance} * \text{Rs.50}$

V. Check whether free delivery applies.

$\text{IF food Subtotal} > \text{Rs.5000}$

$\text{Delivery Charge} = \text{Rs. 0}$

VI. Apply order type discount.

VII. Calculate Discount Amount

VIII. Calculate Final Amount.

$$\text{Final Amount} = \text{Food Subtotal} - \text{Discount} + \text{Delivery Charge}$$

IX. Display the customer bill.

1.4 Outputs

The program displays:

- Customer Name
- Order Type
- Food Subtotal
- Discount Amount
- Price per Item
- Delivery Rate
- Final Payable Amount

1.5 Assumptions

- All prices are entered in Sri Lankan Rupees.
- Customer enters only one order at a time.
- Order type must be Regular, Premium, and VIP.
- Delivery distance is measured in kilometers.
- Discount percentages remain fixed.
- Maximum delivery distance is 20 km.

1.6 Constrains

- Delivery distance cannot exceed 20 km.
- Number of food items must be greater than zero.
- Price per item must be greater than zero.
- Delivery distance cannot be negative.
- Invalid order types are rejected.

Order Type	Discount
Regular	0%
Premium	10%
VIP	20%

- Monetary values are displayed with two decimal places.

2. Algorithm Design.

2.1 Algorithm

Writing the algorithm helped structure business rules and execution flow from inputs to final billing, independent of Python syntax.

1. Start the program.
2. Define global constants:
 - Delivery_Rate = 50
 - Free_Delivery_Limit = 5000
 - Max_Distance = 20.
3. Define Function calculate_discount (order_type, food_subtotal):
 - If order_type is "PREMIUM", return 10% of food_subtotal.
 - Else if order_type is "VIP", return 20% of food_subtotal.
 - Else, return 0.
4. Define Function calculate_delivery(food_subtotal, distance):
 - If food_subtotal > Free_Delivery_Limit, Return 0.
 - Else, Return Distance * Delivery_Rate.
5. Display main title header.
6. Prompt and read customer_name.
7. Input Validation Loop for Order Type:
 - Prompt for order_type.
 - Convert to uppercase.
 - If order_type is in ["REGULAR", "PREMIUM", "VIP"], break loop.

- Else, display error message and repeat.
8. Input Validation Loop for Number of Items:
- If `number_of_items > 0`, break loop.
 - Else, display error message and repeat.
9. Input Validation Loop for Price per Item:
- If `price_per_item > 0`, break loop.
 - Else, display error message and repeat.
10. Input Validation Loop for Delivery Distance:
- If `0 <= distance <= 20 km`, break loop.
 - Else, display error message and repeat.
11. Input Validation Loop for Special Discount:
- Prompt for eligibility (Y/N).
 - If eligibility is in ["Y", "N"], break loop.
 - Else, display error message and repeat.
12. Calculate Costs:
- `food_subtotal = number_of_items * price_per_item`
 - `discount = Call calculate_discount(order_type, food_subtotal)`
 - If `eligibility == "Y"`, add (5% of `food_subtotal`) to `discount`
 - `delivery_fee = Call calculate_delivery(food_subtotal, distance)`
 - `total_payable = food_subtotal + delivery_fee - discount`
13. Display itemized bill formatted to two decimal places.
14. End.

2.2 Psuedocode

My pseudocode code connected the entire logical process with the implementation using Python code. The pseudocode helped me to structure my code process earlier, which made it much easier and efficient to do in the end.

Start

Input Customer_Name, Order_Type, Quantity, Unit_Price, Distance

Validate Input

Subtotal = Quantity * Unit_Price

Discount = Calculate_Discount (Order_Type, Subtotal)

Delivery = Calculate_Delivery (Distance, Subtotal)

Total = Subtotal - Discount + Delivery

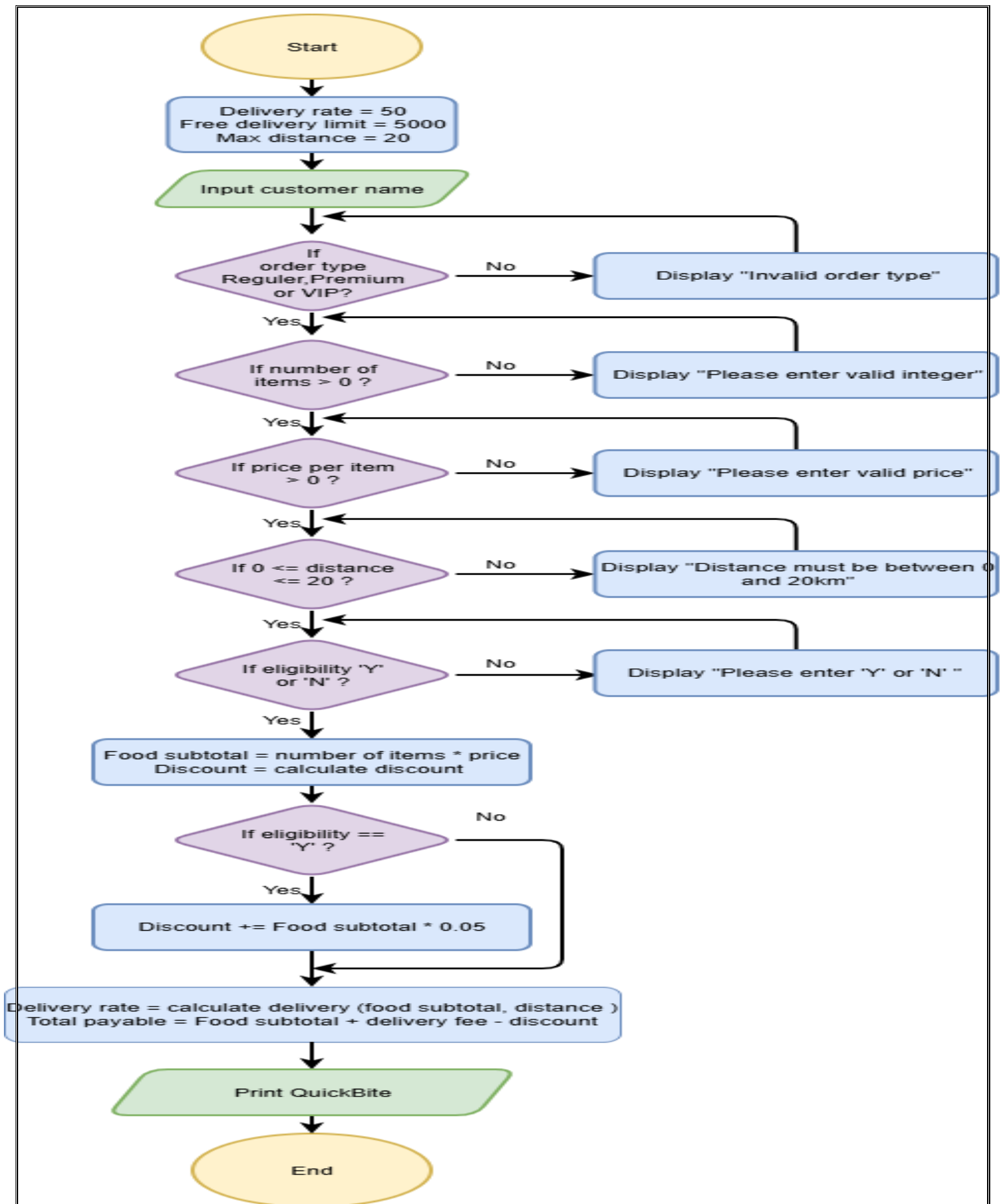
Print Receipt

End

2.3 Flowchart

These standard symbols for flowcharts have been used to represent the logic in the visual form:

- Terminator: Represent program Start/Stop.
- Input/output: Represent Inputs and Receipt Outputs.
- Process: Perform mathematical computations and Function Call.
- Decision: Checks conditional statements and validation.



4. Program

Here is the explanation of the QuickBite Python Program logic in simple English, broken down step-by-step:

4.1 Constants and Functions.

Constants (Fixed Values):

- delivery_rate = 50 (Rs. 50 per kilometer)
- free_delivery_limit = 5000 (Free delivery for orders above Rs. 5000)
- max_distance = 20 (Maximum allowable delivery distance is 20 km)

```
1  #.....
2  #QuickBite Food Ordering System
3  #CCU1304 Fundamentals of Programming
4  #Sample Solution
5  #.....
6
7  delivery_rate=50
8  free_delivery_limit=5000
9  max_distance=20
10
11
12  #.....
13  #Functional to calculate discount
14  #.....
15
16  def calculate_discount(order_type,food_subtotal):
17      if order_type=="PREMIUM":
18          return food_subtotal*0.10
19      elif order_type=="VIP":
20          return food_subtotal*0.20
21      else:
22          return 0
23
```

Functions:

- `calculate_discount()`: Calculates the discount based on the customer category:
 - **PREMIUM**: 20% discount
 - **VIP**: 10% discount
 - **REGULAR**: 0% discount
- `calculate_delivery ()`: Returns 0 if the total order exceeds Rs. 5000; otherwise, calculates Distance * 50.

```
45 #Order Type Validation
46 while True:
47     order_type=input("Enter Order Type(Regular/Premium/VIP):").upper()
48
49     if order_type in["REGULAR","PREMIUM","VIP"]:
50         break
51     else:
52         print("Invalid order type.Please try again.")
53
54 #Number of Items Validation
55 while True:
56     try:
57         number_of_items=int(input("Enter number of food items:"))
58         if number_of_items>0:
59             break
60         else:
61             print("Number of items must be greater than zero.")
62     except:
63         print("Please enter a valid integer.")
64
```

4.2 User Inputs and Data Validation.

The program asks the user for necessary information and uses WHILE loops to prevent invalid inputs:

- **Customer Name**: Accepts the customer's name.
- **Order Type**: Validates that the user enters only "REGULAR", "PREMIUM", or "VIP". If invalid, it prompts again.
- **Number of Items**: Checks that the quantity entered is a number greater than 0.

- **Price per Item:** Validates that the price is greater than 0.
- **Delivery Distance:** Ensures the distance is between 0 and 20 km.
- **Special Discount Eligibility:** Validates input to accept only 'Y' or 'N'.

```

76 #Delivery Distance Validation
77 while True:
78     try:
79         distance=float(input("Enter delivery distance(km):"))
80
81         if 0<=distance<=max_distance:
82             break
83         else:
84             print("Delivery distance must be between 0 and 20 km.")
85     except:
86         print("Please enter a valid distance.")
87
88 #Discount Eligibility
89 while True:
90     eligibility=input("Eligible for Special Discount?(Y/N):").upper()
91
92     if eligibility in["Y","N"]:
93         break
94     else:
95         print("Please enter Y or N.")
96

```

4.3 Calculations

Using the validated input, the program calculates the final bill components:

1. **Food Subtotal** = Number of Items * Price per Item
2. **Discount** = calculate_discount() + (An additional 5% discount if special eligibility is 'Y')
3. **Delivery Charge** = calculate_delivery()
4. **Total Payable** = Food Subtotal + Delivery Charge – Discount

```

97  #.....
98  #Calculations
99  #.....
100
101  food_subtotal=number_of_items*price_per_item
102
103  discount=calculate_discount(order_type,food_subtotal)
104
105  #Additinal promotional discount
106  if eligibility=="Y":
107      |    discount+=food_subtotal*0.05
108
109  delivery_rate=calculate_delivery(food_subtotal,distance)
110
111  total_payable=food_subtotal+delivery_rate-discount
112

```

4.4 Generating the Itemized Receipt

Finally, the program formats all calculated values to two decimal places and displays a clear, detailed bill on the screen.

```

117  print("\n")
118  print("="*50)
119  print("QUICKBITE ORDER BILL")
120  print("="*50)
121
122  print(f"Customer Name      : {customer_name}")
123  print(f"Order Type           : {order_type.title()}")
124  print(f"Food Items             : {number_of_items}")
125  print(f"Price Per Item          : Rs.{price_per_item:.2f}")
126  print(f"Food Subtotal           : Rs.{food_subtotal:.2f}")
127  print(f"Discount                : Rs.{discount:.2f}")
128  print(f"Delivery_rate           : Rs.{delivery_rate:.2f}")
129
130  print("="*50)
131  print(f"TOTAL PAYABLE           : Rs.{total_payable:.2f}")
132  print("="*50)
133
134  print("\nThank you for ordering from QuickBite!")
135

```

5. Input Validations

Input Validation is the process of checking the data entered by the user to ensure it is correct, safe, and logical before the program processes it.

If the user enters invalid data, input validation catches the error, displays a helpful message, and asks the user to re-enter the data.

5.1 How Input Validation Works in this Program

5.1.1 Order Type Validation

- The order type must be either **REGULAR**, **PREMIUM**, or **VIP**.
- Converts the input to uppercase using `.upper()` and checks if it exists in the list of allowed types.

```
45 #Order Type Validation
46 while True:
47     order_type=input("Enter Order Type(Regular/Premium/VIP):").upper()
48
49     if order_type in["REGULAR","PREMIUM","VIP"]:
50         break
51     else:
52         print("Invalid order type.Please try again.")
53
```

- **Correct Case:** Accepts only valid order categories: REGULAR, PREMIUM, or VIP (handles both uppercase and lowercase inputs).

```
=====
QUICKBITE FOOD ORDER SYSREM
=====
Enter customer name:Oshadhi
Enter Order Type(Regular/Premium/VIP):Vip
Enter number of food items:█
```

- **Error Case:** If an unsupported option is entered, the system displays "Invalid order type. Please try again."

```
=====
QUICKBITE FOOD ORDER SYSREM
=====
Enter customer name:Oshadhi
Enter Order Type(Regular/Premium/VIP):Gold123
Invalid order type.Please try again.
Enter Order Type(Regular/Premium/VIP):
```

5.1.2 Number of Items Validation

- Uses try-except to prevent program crashes if text is typed instead of a number, and checks if items > 0.

```
54 #Number of Items Validation
55 while True:
56     try:
57         number_of_items=int(input("Enter number of food items:"))
58         if number_of_items>0:
59             break
60         else:
61             print("Number of items must be greater than zero.")
62     except:
63         print("Please enter a valid integer.")
64
```

- **Correct Case:** Accepts any positive whole number greater than 0.

```
=====
QUICKBITE FOOD ORDER SYSREM
=====
Enter customer name:Oshadhi
Enter Order Type(Regular/Premium/VIP):Premium
Enter number of food items:3
Enter price per item(Rs.):
```


- **Error Cases:**

- Entering text triggers a Value Error catch block, prompting "Please enter a valid integer".

```
=====
QUICKBITE FOOD ORDER SYSREM
=====
Enter customer name:Oshadhi
Enter Order Type(Regular/Premium/VIP):Premium
Enter number of food items:two
Please enter a valid integer.
Enter number of food items:[]
```

5.1.3 Price per Item Validation

- Uses float () conversion and checks if price > 0.

```
65 #Price Validation
66 while True:
67     try:
68         price_per_item=float(input("Enter price per item(Rs.):"))
69         if price_per_item>0:
70             break
71         else:
72             print("Price must be greater than zero.")
73     except:
74         print("Please enter a valid price.")
75
```

- **Correct Case:** Accepts positive numeric values including decimals.

```
=====
QUICKBITE FOOD ORDER SYSREM
=====
Enter customer name:Oshadhi
Enter Order Type(Regular/Premium/VIP):Vip
Enter number of food items:4
Enter price per item(Rs.):150.50
Enter delivery distance(km):[]
```

- **Error Cases:**

- Typing letters or symbols is displaying "Please enter a valid price".
- Entering zero or a negative price prompts "Price must be greater than zero".

```
=====
QUICKBITE FOOD ORDER SYSREM
=====
Enter customer name:Oshadhi
Enter Order Type(Regular/Premium/VIP):Vip
Enter number of food items:6
Enter price per item(Rs.):500
Price must be greater than zero.
Enter price per item(Rs.):abc
Please enter a valid price.
Enter price per item(Rs.):
```

5.1.4 Delivery Distance Validation

- Checks if $0 \leq \text{distance} \leq 20$.

```
76 #Delivery Distance Validation
77 while True:
78     try:
79         distance=float(input("Enter delivery distance(km):"))
80
81         if 0<=distance<=max_distance:
82             break
83         else:
84             print("Delivery distance must be between 0 and 20 km.")
85     except:
86         print("Please enter a valid distance.")
87
```

- **Correct Case:** Accepts values within the allowed range of **0 km to 20 km**.

```
=====
QUICKBITE FOOD ORDER SYSREM
=====
Enter customer name:Oshadhi
Enter Order Type(Regular/Premium/VIP):Regular
Enter number of food items:5
Enter price per item(Rs.):100
Enter delivery distance(km):5.5
Eligible for Special Discount?(Y/N):
```

- **Error Cases:** Entering a value greater than the max distance displays "Distance must be between 0 and 20 km".

```
=====
QUICKBITE FOOD ORDER SYSREM
=====
Enter customer name:Oshadhi
Enter Order Type(Regular/Premium/VIP):Vip
Enter number of food items:8
Enter price per item(Rs.):200
Enter delivery distance(km):-5
Delivery distance must be between 0 and 20 km.
Enter delivery distance(km):
```

5.1.5 Discount Eligibility Validation

- Validates 'Y' or 'N' inputs after converting to uppercase.

```
88 #Discount Eligibility
89 while True:
90     eligibility=input("Eligible for Special Discount?(Y/N):").upper()
91
92     if eligibility in["Y","N"]:
93         break
94     else:
95         print("Please enter Y or N.")
96
```

- **Correct Case:** Accepts either 'Y' or 'N'.

```
=====
QUICKBITE ORDER BILL
=====
Customer Name      : Oshadhi
Order Type         : Premium
Food Items         : 7
Price Per Item     : Rs.1000.00
Food Subtotal      : Rs.7000.00
Discount           : Rs.1050.00
Delivery_rate      : Rs.0.00
=====
TOTAL PAYABLE      : Rs.5950.00
=====

Thank you for ordering from QuickBite!
```

- **Error Case:** Typing any other character or number displays " Please enter 'Y' or 'N'" and re-prompts the use

```
=====
QUICKBITE FOOD ORDER SYSREM
=====
Enter customer name:Oshadhi
Enter Order Type(Regular/Premium/VIP):Vip
Enter number of food items:5
Enter price per item(Rs.):500
Enter delivery distance(km):7
Eligible for Special Discount?(Y/N):X
Please enter Y or N.
Eligible for Special Discount?(Y/N):
```

6. Testing

6.1 Test Plan

Testing ensures that QuickBite functions correctly under normal, boundary, and invalid test cases, giving appropriate results and error checking.

Objectives:

- To validate calculations of food costs, delivery charges, and Premium/VIP discounts.
- To ensure that free delivery is given for orders greater than Rs. 5000.
- To validate input validation and invalid data handling.

6.2 Test Cases

Test ID	Test Scenario	Input	Expected Result	Status
TC01	Normal Regular Order	Regular, 5 items, Rs.500, 5km	Total calculated correctly	Pass
TC02	Normal Premium Order	Premium, 4 items, Rs.700, 4km	10% discount applied.	Pass
TC03	Normal VIP Order	VIP, 5 items, Rs.1000, 3km	20% discount applied.	Pass
TC04	Free Delivery	VIP, 6 items, Rs.1000, 8km	Delivery charge = Rs.0	Pass
TC05	Boundary Distance	Distance = 20km	Accepted successfully	Pass
TC06	Invalid Distance	Distance = 25km	Display validation error	Pass
TC07	Negative Items	Items = -2	Display validation error	Pass
TC08	Invalid Order Type	Gold	Display invalid order type message	Pass

7. Test Evidence

TC01

- Normal Regular order output verification.

```
=====
QUICKBITE ORDER BILL
=====
Customer Name      : Oshadhi
Order Type         : Regular
Food Items         : 5
Price Per Item     : Rs.500.00
Food Subtotal      : Rs.2500.00
Discount           : Rs.0.00
Delivery_rate      : Rs.250.00
=====
TOTAL PAYABLE      : Rs.2750.00
=====

Thank you for ordering from QuickBite!
PS C:\Users\User\OneDrive\Desktop> 
```

TC02

- Validated 10% discount for Premium membership.

```
=====
QUICKBITE ORDER BILL
=====
Customer Name      : Oshadhi
Order Type         : Premium
Food Items         : 4
Price Per Item     : Rs.700.00
Food Subtotal      : Rs.2800.00
Discount           : Rs.420.00
Delivery_rate      : Rs.200.00
=====
TOTAL PAYABLE      : Rs.2580.00
=====
```

TC03

- Validated 20% discount for VIP membership.

```
=====
QUICKBITE ORDER BILL
=====
Customer Name      : Oshadhi
Order Type         : Vip
Food Items         : 5
Price Per Item     : Rs.1000.00
Food Subtotal      : Rs.5000.00
Discount           : Rs.1250.00
Delivery_rate      : Rs.150.00
=====
TOTAL PAYABLE      : Rs.3900.00
=====
```

TC04

- Verified free delivery waiver for orders > Rs.5000

```
=====
QUICKBITE ORDER BILL
=====
Customer Name      : Oshadhi
Order Type         : Vip
Food Items         : 6
Price Per Item     : Rs.1000.00
Food Subtotal      : Rs.6000.00
Discount           : Rs.1500.00
Delivery_rate      : Rs.0.00
=====
TOTAL PAYABLE      : Rs.4500.00
=====

Thank you for ordering from QuickBite!
```


TC05

- Boundary test at 20km maximum delivery range.

```
=====
QUICKBITE ORDER BILL
=====
Customer Name      : Oshadhi
Order Type         : Premium
Food Items         : 4
Price Per Item     : Rs.800.00
Food Subtotal      : Rs.3200.00
Discount           : Rs.320.00
Delivery_rate      : Rs.1000.00
=====
TOTAL PAYABLE      : Rs.3880.00
=====

Thank you for ordering from QuickBite!
```

TC06

- Error prompt displayed for invalid distance (>20km).

```
=====
QUICKBITE FOOD ORDER SYSREM
=====
Enter customer name:Oshadhi
Enter Order Type(Regular/Premium/VIP):VIP
Enter number of food items:8
Enter price per item(Rs.):200
Enter delivery distance(km):25
Delivery distance must be between 0 and 20 km.
```

TC07

- Error prompt displayed for negative item quantity.

```
=====
QUICKBITE FOOD ORDER SYSREM
=====
Enter customer name:Oshadhi
Enter Order Type(Regular/Premium/VIP):Premium
Enter number of food items:-2
Number of items must be greater than zero.
Enter number of food items:[]
```

TC08

- Error prompt displayed for invalid order tier.

```
=====
QUICKBITE FOOD ORDER SYSREM
=====
Enter customer name:Oshadhi
Enter Order Type(Regular/Premium/VIP):Gold
Invalid order type.Please try again.
Enter Order Type(Regular/Premium/VIP):[]
```

8. Technical Documentation.

QuickBite is built in Python using procedural programming, using modular functions to improve readability, maintainability, and code reuse.

8.1. Main Functions.

1. Validate input ()

This function validates user inputs such as:

- Number of items
- Price per item
- Delivery distance

Invalid entries trigger an error message and prompt the user to retry.

2. Calculate bill ()

This function performs all calculations.

It calculates:

- Food subtotal
- Discount amount
- Delivery charge
- Final payable amount

3. Display bill ()

This function prints the final itemized bill neatly with:

- Customer details
- Food cost
- Delivery charge
- Final total

8.2 Variables Used

Variable	Data Type	Purpose
customer_name	String	Stores customer name
order_type	String	Stores customer category
num_items	Integer	Stores quantity of food items
price_per_item	Float	Stores unit price
distance	Float	Delivery distance
food_total	Float	Total food cost
discount	Float	Discount amount
delivery_charge	Float	Delivery fee
total	Float	Final payable amount

8.3 Programming Decisions.

Key development decisions made the program maintainable and easy to modify:

- Functions: Avoid code repetition.
- Input Validation: Blocks invalid data.
- If-Else Logic: Handles tier discounts.
- Arithmetic Operators: Compute order totals.
- Constants & Clear Naming: Improve readability (e.g., delivery rate of Rs. 50/km).

9. Reflection.

Building QuickBite was a very useful learning process about Python fundamentals such as functions, loops, and validation. The difficulties were mainly in making sure that the discount and delivery conditions were implemented right and also handling the erroneous input via while and try-except.

9.1 Future Enhancements.

Such enhancements can be considered as GUI development, incorporation of the database with history of orders, payment method expansion, and loyalty reward program.

